Pipeline MNIST en TensorFlow

Guía académica 03 · De los píxeles a la evaluación multiclase

Juan Sebastián Laverde Chunza

15 de octubre de 2026

Esta guía recorre un proyecto completo de clasificación de imágenes con TensorFlow y Keras: - carga de MNIST con `tensorflow_datasets` y exploración; - pesos de clase y un *pipeline* de `tf.data`; - un modelo denso con normalización por lotes; - entrenamiento con *callbacks*, elección del mejor modelo y evaluación final con matriz de confusión, F1 macro y análisis de errores.

Todo el código se ejecutó al generar el documento, y las cifras y figuras son reales.

Tabla de contenidos

1 Propósito y mapa de lectura **2**

2 Intuición: enseñar a reconocer garabatos **2**

3 Fundamentos matemáticos **3**
- 3.1 De diez puntuaciones a diez probabilidades . . . 3
- 3.2 Entropía cruzada categórica . . . 3
- 3.3 Ejemplo numérico resuelto . . . 3
- 3.4 Pesos de clase . . . 3
- 3.5 Métricas multiclase . . . 4

4 Datos: carga, exploración y partición **4**
- 4.1 Carga con `tensorflow_datasets` . . . 4
- 4.2 Exploración . . . 5
- 4.3 Balance de clases y pesos . . . 6

5 Flujo de entrada con `tf.data` **6**

6 Modelo: red densa con normalización por lotes **7**

7 Entrenamiento **8**
- 7.1 Compilar . . . 8
- 7.2 *Callbacks* . . . 8

8 Evaluación final **10**
- 8.1 El mejor modelo frente al conjunto de prueba . . . 10
- 8.2 Predicciones y matriz de confusión . . . 10
- 8.3 Métricas por clase y confusiones frecuentes . . . 12
- 8.4 Análisis de errores . . . 12

9 Dimensiones de tensores y flujo de datos **13**

10 Hiperparámetros del modelo **15**

11 Errores frecuentes **15**

12 Ejercicios **16**
12.1 Respuestas orientativas 16

13 Glosario **16**

14 Conclusiones académicas **17**

15 Referencias **18**

Procedencia y autoría

Autor de esta guía. Juan Sebastián Laverde Chunza, con apoyo de herramientas de inteligencia artificial generativa. Es material académico complementario y **no** es material oficial del docente ni de la Universidad El Bosque.
Referencia académica. Los temas siguen el notebook `Pipeline_TensorFlow.ipynb` de la Clase 02 del curso Deep Learning I, del profesor **Héctor Javier Hortúa, PhD** (GitHub · Colab).
Edición pública. El código, las particiones, los ejemplos y las explicaciones de esta edición son propios e independientes: no reproducen el código ni el texto del notebook del docente. MNIST es un conjunto de datos público (LeCun, Cortes y Burges). Citar el notebook como referencia no equivale a una licencia ni a una autorización sobre el material original.

1. Propósito y mapa de lectura

Un *pipeline* de aprendizaje automático es la cadena completa que va de los datos crudos a una decisión evaluada. Las etapas de esta guía son:

1. **Datos** (§4): cargar, explorar, dividir sin fugas de información y calcular pesos de clase.
2. **Flujo de entrada** (§5): servir lotes de forma eficiente con `tf.data`.
3. **Modelo** (§6): una red densa con normalización por lotes y *Dropout*.
4. **Entrenamiento** (§7): compilar, entrenar con *callbacks* y leer el historial.
5. **Evaluación** (§8): medir en prueba con el mejor modelo, construir la matriz de confusión y analizar errores.

2. Intuición: enseñar a reconocer garabatos

Pide a alguien que nunca ha visto números escritos a mano que aprenda a reconocerlos con miles de ejemplos etiquetados. Al principio adivina. Con cada lote de ejemplos corrige su criterio. Cada cierto tiempo se le hace un examen de práctica con ejemplos nuevos (**validación**), para saber si de verdad generaliza o solo memoriza. Al final presenta un examen definitivo que nunca vio (**prueba**). La matriz de confusión es la corrección de ese examen: muestra qué dígitos confunde con cuáles.

3. Fundamentos matemáticos

3.1. De diez puntuaciones a diez probabilidades

La última capa produce un vector de $K = 10$ ***logits*** $\mathbf{z}$. La función **softmax** lo convierte en una distribución de probabilidad:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_k p_k = 1.$$

La clase predicha es $\hat{y} = \arg\max_k p_k = \arg\max_k z_k$, porque softmax conserva el orden.

3.2. Entropía cruzada categórica

Si la etiqueta verdadera se codifica *one-hot* como $\mathbf{y}$, con un 1 en la clase correcta c:

$$\ell(\mathbf{z}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log p_k = -\log p_c.$$

Solo importa la probabilidad asignada a la clase correcta. Su gradiente respecto a los logits es $\partial \ell / \partial z_k = p_k - y_k$, la generalización directa del caso binario.

Variante en Keras	Formato de la etiqueta	Entrada de la pérdida
`CategoricalCrossentropy`	*One-hot*, forma `(N, K)`	Probabilidades o logits (`from_logits=True`)
`SparseCategoricalCrossentropy`	Entero, forma `(N,)`	Probabilidades o logits (`from_logits=True`)

3.3. Ejemplo numérico resuelto

Con tres clases y logits $\mathbf{z} = (2.0,\ 0.5,\ -1.0)$:

- $e^{\mathbf{z}} = (7.389,\ 1.649,\ 0.368)$, con suma 9.406.
- $\mathbf{p} = (0.786,\ 0.175,\ 0.039)$.
- Si la clase correcta es la primera: $\ell = -\log 0.786 = 0.241$.
- Si es la tercera: $\ell = -\log 0.039 = 3.24$. Una confianza equivocada cuesta trece veces más.
- Gradiente cuando la correcta es la primera: $\mathbf{p} - \mathbf{y} = (-0.214,\ 0.175,\ 0.039)$. Sube el logit correcto y baja los demás.

3.4. Pesos de clase

Si las clases tienen frecuencias n_c distintas, el modelo tiende a favorecer las mayoritarias. Una corrección habitual pondera la pérdida de cada ejemplo con

$$w_c = \frac{N}{K\, n_c},$$

donde N es el número total de ejemplos de entrenamiento. Si todas las clases tuvieran $n_c = N/K$, todos los pesos valdrían 1. Por ejemplo, con $N = 1000$, $K = 2$, $n_0 = 800$ y $n_1 = 200$: $w_0 = 0.625$ y $w_1 = 2.5$. Cada ejemplo minoritario pesa cuatro veces más.

3.5. Métricas multiclase

Para cada clase k, con verdaderos positivos VP_k, falsos positivos FP_k y falsos negativos FN_k:

$$\text{precisión}_k = \frac{VP_k}{VP_k + FP_k}, \quad \text{sensibilidad}_k = \frac{VP_k}{VP_k + FN_k}, \quad F1_k = \frac{2\,\text{prec}_k\,\text{sens}_k}{\text{prec}_k + \text{sens}_k}.$$

El **F1 macro** es el promedio simple $\frac{1}{K}\sum_k F1_k$: todas las clases cuentan igual, sin importar su tamaño. Por eso detecta si el modelo funciona mal en alguna clase aunque la exactitud global sea alta.

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"

import tempfile, pathlib
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import tensorflow_datasets as tfds
import keras

tfds.disable_progress_bar()
keras.utils.set_random_seed(1234)
print("TensorFlow", tf.__version__, "| Keras", keras.__version__,
      "| TFDS", tfds.__version__)
```

```
TensorFlow 2.21.0 | Keras 3.15.1 | TFDS 4.9.10
```

```
logits = tf.constant([[2.0, 0.5, -1.0]])
print("softmax:", tf.nn.softmax(logits).numpy().round(3))
for true_class in (0, 2):
    target = tf.one_hot([true_class], depth=3)
    loss_fn = keras.losses.CategoricalCrossentropy(from_logits=True)
    loss = loss_fn(target, logits)
    print(f"clase correcta {true_class}: perdida = {float(loss):.3f}")
```

```
softmax: [[0.786 0.175 0.039]]
clase correcta 0: perdida = 0.241
clase correcta 2: perdida = 3.241
```

4. Datos: carga, exploración y partición

4.1. Carga con `tensorflow_datasets`

TFDS descarga MNIST una sola vez, lo guarda en caché y lo entrega como `tf.data.Dataset`. La sintaxis de *slicing* de particiones permite crear subconjuntos sin copiar datos. Esta guía usa:

- **Entrenamiento:** el 85 % inicial del *train* original (51 000 imágenes).
- **Validación:** el 15 % restante (9 000 imágenes).
- **Prueba:** el *test* original completo (10 000 imágenes), que solo se usa al final.

```
(raw_fit, raw_check, raw_exam), info = tfds.load(
    "mnist",
    split=["train[:85%]", "train[85%:]", "test"],
    as_supervised=True,
    with_info=True,
)
n_classes = info.features["label"].num_classes
sizes = {name: info.splits[s].num_examples for name, s in
         [("entrenamiento", "train[:85%]"), ("validacion", "train[85%:]"),
          ("prueba", "test")]}
print("forma de imagen:", info.features["image"].shape,
      "| clases:", n_classes)
for name, size in sizes.items():
    print(f"  {name:<14} {size:>6} imagenes")
```

```
forma de imagen: (28, 28, 1) | clases: 10
  entrenamiento   51000 imagenes
  validacion       9000 imagenes
  prueba          10000 imagenes
```

4.2. Exploración

Antes de modelar conviene mirar los datos: rango de valores, tipo, forma y balance de clases.

```
image, label = next(iter(raw_fit))
print("tipo:", image.dtype, "| forma:", tuple(image.shape),
      "| min/max:", int(tf.reduce_min(image)), int(tf.reduce_max(image)),
      "| etiqueta:", int(label))
```

```
tipo: <dtype: 'uint8'> | forma: (28, 28, 1) | min/max: 0 255 | etiqueta: 4
```

```
fig, axes = plt.subplots(2, 6, figsize=(7.2, 2.7))
for ax, (img, lab) in zip(axes.ravel(), raw_fit.take(12)):
    ax.imshow(img[..., 0], cmap="gray_r")
    ax.set_title(int(lab), fontsize=9); ax.axis("off")
plt.tight_layout(); plt.show()
```

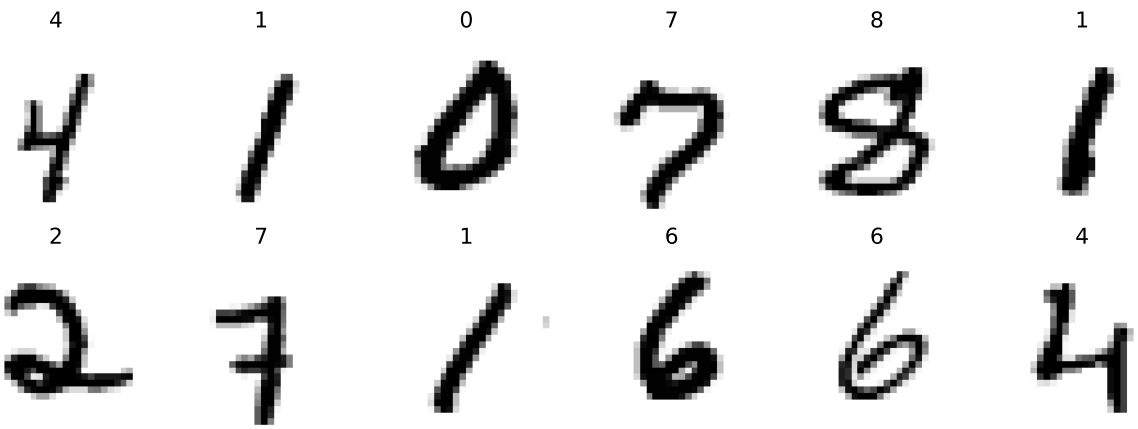


Figura 1: Doce imágenes de entrenamiento con su etiqueta.

4.3. Balance de clases y pesos

Se cuentan las etiquetas **solo en el conjunto de entrenamiento**. Usar validación o prueba para calcular cualquier estadística filtraría información de datos que el modelo no debe ver.

```
fit_labels = np.fromiter((int(y) for _, y in raw_fit), dtype=np.int64)
counts = np.bincount(fit_labels, minlength=n_classes)
weights = len(fit_labels) / (n_classes * counts)
class_weight = {k: float(w) for k, w in enumerate(weights)}
for k in range(n_classes):
    print(f"digito {k}: n={counts[k]:>5}  peso={weights[k]:.3f}")
```

```
digito 0: n= 5050  peso=1.010
digito 1: n= 5723  peso=0.891
digito 2: n= 5065  peso=1.007
digito 3: n= 5215  peso=0.978
digito 4: n= 4945  peso=1.031
digito 5: n= 4599  peso=1.109
digito 6: n= 5021  peso=1.016
digito 7: n= 5319  peso=0.959
digito 8: n= 4981  peso=1.024
digito 9: n= 5082  peso=1.004
```

MNIST está casi balanceado: los pesos quedan aproximadamente entre 0.89 y 1.11. Aquí el mecanismo tiene un efecto pequeño, pero es el mismo que se aplicaría a un conjunto muy desbalanceado, como uno de detección de fraude.

5. Flujo de entrada con `tf.data`

Cada transformación tiene una razón de ser, y el orden importa:

Paso	Qué hace	Por qué en esta posición
`map(prepare)`	Convierte a `float32` en $[0, 1]$ y la etiqueta a *one-hot*	Transformación determinista por ejemplo
`cache()`	Guarda el resultado en memoria tras la primera época	Después del `map` costoso y antes de lo aleatorio
`shuffle(n)`	Mezcla con un búfer de n ejemplos	Después de `cache`, para que el orden cambie en cada época
`batch(b)`	Agrupa b ejemplos	Después de mezclar, para que los lotes varíen
`prefetch(AUTOTUNE)`	Prepara el siguiente lote en paralelo	Siempre al final

```
BATCH = 128

def prepare(img, lab):
    img = tf.cast(img, tf.float32) / 255.0
    return img, tf.one_hot(lab, n_classes)
```

```
def build_stream(ds, training):
    ds = ds.map(prepare, num_parallel_calls=tf.data.AUTOTUNE).cache()
    if training:
        ds = ds.shuffle(10_000, seed=99, reshuffle_each_iteration=True)
    return ds.batch(BATCH).prefetch(tf.data.AUTOTUNE)

fit_stream = build_stream(raw_fit, training=True)
check_stream = build_stream(raw_check, training=False)
exam_stream = build_stream(raw_exam, training=False)

xb, yb = next(iter(fit_stream))
print("lote X:", tuple(xb.shape), xb.dtype.name,
      "| rango:", float(tf.reduce_min(xb)), "-", float(tf.reduce_max(xb)))
print("lote y:", tuple(yb.shape), "| primera etiqueta one-hot:",
      yb[0].numpy().astype(int))
```

```
lote X: (128, 28, 28, 1) float32 | rango: 0.0 - 1.0
lote y: (128, 10) | primera etiqueta one-hot: [0 0 0 0 0 1 0 0 0 0]
```

6. Modelo: red densa con normalización por lotes

Una red **densa** no conoce la estructura espacial de la imagen. `Flatten` convierte los 28×28 píxeles en un vector de 784 valores, y cada neurona de la primera capa mira todos los píxeles a la vez. Las redes convolucionales aprovechan mejor la geometría, pero una red densa ya alcanza buen desempeño en MNIST y permite concentrarse en el *pipeline*.

Cada bloque oculto sigue el orden `Dense → BatchNormalization → ReLU → Dropout`:

- `Dense` sin sesgo (`use_bias=False`), porque la normalización posterior lo hace redundante: su parámetro β cumple ese papel.
- `BatchNormalization` estabiliza la escala de las activaciones y permite tasas de aprendizaje mayores.
- `Dropout` regulariza.

La capa de salida no tiene activación: entrega logits.

```
def digit_classifier(widths=(256, 128), drop=0.25):
    pixels = keras.Input(shape=(28, 28, 1), name="pixeles")
    h = keras.layers.Flatten()(pixels)
    for i, width in enumerate(widths, start=1):
        h = keras.layers.Dense(width, use_bias=False, name=f"densa_{i}")(h)
        h = keras.layers.BatchNormalization(name=f"bn_{i}")(h)
        h = keras.layers.Activation("relu", name=f"relu_{i}")(h)
        h = keras.layers.Dropout(drop, name=f"dropout_{i}")(h)
    logits = keras.layers.Dense(n_classes, name="logits")(h)
    return keras.Model(pixels, logits, name="clasificador_digitos")

model = digit_classifier()
for layer in model.layers:
    print(f"{layer.name:<12} salida={str(layer.output.shape):<12} "
          f"parametros={layer.count_params()}")
print("total:", model.count_params())
```

```
pixeles      salida=(None, 28, 28, 1) parametros=0
```

```
flatten      salida=(None, 784)  parametros=0
densa_1      salida=(None, 256)  parametros=200704
bn_1         salida=(None, 256)  parametros=1024
relu_1       salida=(None, 256)  parametros=0
dropout_1    salida=(None, 256)  parametros=0
densa_2      salida=(None, 128)  parametros=32768
bn_2         salida=(None, 128)  parametros=512
relu_2       salida=(None, 128)  parametros=0
dropout_2    salida=(None, 128)  parametros=0
logits       salida=(None, 10)   parametros=1290
total: 236298
```

La capa `densa_1` concentra la mayoría de los parámetros: $784 \times 256 = 200\,704$. Cada capa `bn_k` tiene cuatro vectores del tamaño de la capa: γ y β, que se entrenan, y la media y la varianza móviles, que no se entrenan.

7. Entrenamiento

7.1. Compilar

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=2e-3),
    loss=keras.losses.CategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.CategoricalAccuracy(name="acc"),
             keras.metrics.F1Score(average="macro", name="f1_macro")],
)
```

`F1Score(average="macro")` asigna a cada fila la clase de máxima puntuación. Como softmax conserva el orden, funciona igual con logits que con probabilidades.

7.2. *Callbacks*

Callback	Vigila	Acción
`TensorBoard`	Todas las métricas	Escribe registros para visualizarlos con `tensorboard --logdir`
`EarlyStopping`	`val_loss`	Detiene el entrenamiento tras `patience` épocas sin mejora
`ModelCheckpoint`	`val_loss`	Guarda en disco el mejor modelo (`save_best_only=True`)
`ReduceLROnPlateau`	`val_loss`	Multiplica la tasa por `factor` si la mejora se estanca
`TerminateOnNaN`	La pérdida	Corta el entrenamiento si aparece un `NaN`
`CSVLogger`	Todas las métricas	Guarda el historial en un archivo CSV

```
run_dir = pathlib.Path(tempfile.mkdtemp(prefix="guia03_"))
best_path = run_dir / "mejor_modelo.keras"
callbacks = [
    keras.callbacks.TensorBoard(log_dir=str(run_dir / "tb")),
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=4),
    keras.callbacks.ModelCheckpoint(best_path, monitor="val_loss",
                                    save_best_only=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.3,
                                      patience=2, min_lr=1e-5),
    keras.callbacks.TerminateOnNaN(),
    keras.callbacks.CSVLogger(str(run_dir / "historial.csv")),
]
history = model.fit(fit_stream, validation_data=check_stream, epochs=25,
                    class_weight=class_weight, callbacks=callbacks,
                    verbose=0)
h = history.history
for ep in range(len(h["loss"])):
    print(f"epoca {ep + 1:>2}  loss={h['loss'][ep]:.4f}  "
          f"val_loss={h['val_loss'][ep]:.4f}  "
          f"val_acc={h['val_acc'][ep]:.4f}  lr={h['learning_rate'][ep]:.1e}")
```

```
epoca  1  loss=0.2936  val_loss=0.1401  val_acc=0.9594  lr=2.0e-03
epoca  2  loss=0.1430  val_loss=0.1057  val_acc=0.9687  lr=2.0e-03
epoca  3  loss=0.1120  val_loss=0.0879  val_acc=0.9737  lr=2.0e-03
epoca  4  loss=0.0908  val_loss=0.0810  val_acc=0.9749  lr=2.0e-03
epoca  5  loss=0.0797  val_loss=0.0825  val_acc=0.9751  lr=2.0e-03
epoca  6  loss=0.0689  val_loss=0.0791  val_acc=0.9764  lr=2.0e-03
epoca  7  loss=0.0613  val_loss=0.0772  val_acc=0.9773  lr=2.0e-03
epoca  8  loss=0.0570  val_loss=0.0755  val_acc=0.9788  lr=2.0e-03
epoca  9  loss=0.0504  val_loss=0.0743  val_acc=0.9804  lr=2.0e-03
epoca 10  loss=0.0465  val_loss=0.0698  val_acc=0.9817  lr=2.0e-03
epoca 11  loss=0.0438  val_loss=0.0707  val_acc=0.9813  lr=2.0e-03
epoca 12  loss=0.0425  val_loss=0.0685  val_acc=0.9818  lr=2.0e-03
epoca 13  loss=0.0397  val_loss=0.0733  val_acc=0.9799  lr=2.0e-03
epoca 14  loss=0.0371  val_loss=0.0759  val_acc=0.9811  lr=2.0e-03
epoca 15  loss=0.0273  val_loss=0.0650  val_acc=0.9838  lr=6.0e-04
epoca 16  loss=0.0225  val_loss=0.0621  val_acc=0.9843  lr=6.0e-04
epoca 17  loss=0.0206  val_loss=0.0596  val_acc=0.9850  lr=6.0e-04
epoca 18  loss=0.0168  val_loss=0.0609  val_acc=0.9860  lr=6.0e-04
epoca 19  loss=0.0174  val_loss=0.0619  val_acc=0.9848  lr=6.0e-04
epoca 20  loss=0.0146  val_loss=0.0606  val_acc=0.9852  lr=1.8e-04
epoca 21  loss=0.0129  val_loss=0.0594  val_acc=0.9858  lr=1.8e-04
epoca 22  loss=0.0127  val_loss=0.0604  val_acc=0.9852  lr=1.8e-04
epoca 23  loss=0.0124  val_loss=0.0605  val_acc=0.9858  lr=1.8e-04
epoca 24  loss=0.0128  val_loss=0.0608  val_acc=0.9862  lr=5.4e-05
epoca 25  loss=0.0124  val_loss=0.0603  val_acc=0.9861  lr=5.4e-05
```

```
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.8))
axes[0].plot(h["loss"], label="entrenamiento")
axes[0].plot(h["val_loss"], label="validación")
axes[0].set_xlabel("época"); axes[0].set_ylabel("entropía cruzada")
axes[0].legend()
axes[1].plot(h["acc"], label="entrenamiento")
```

```
axes[1].plot(h["val_acc"], label="validación")
axes[1].set_xlabel("época"); axes[1].set_ylabel("exactitud")
plt.tight_layout(); plt.show()
```

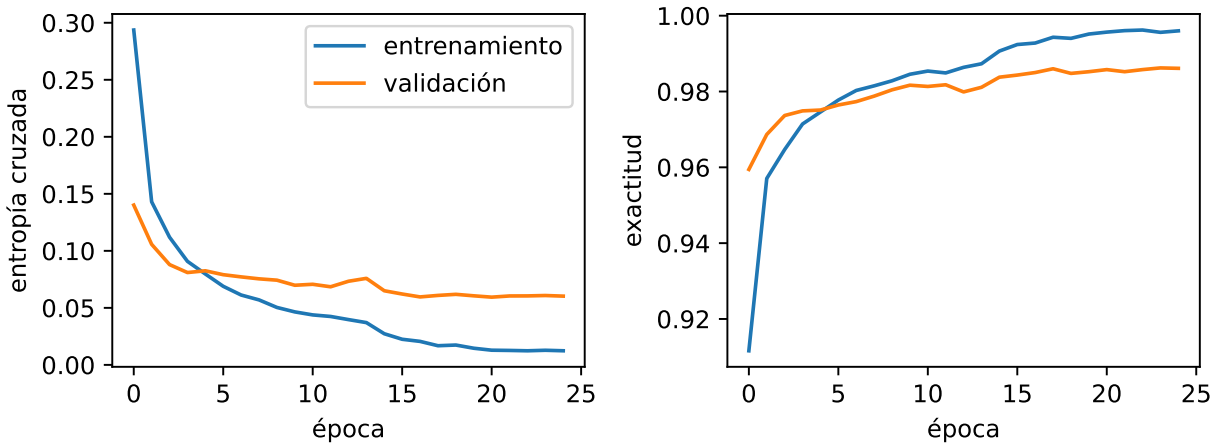


Figura 2: Historial: pérdida y exactitud por época en entrenamiento y validación.

Cómo leer el historial

En las primeras épocas la exactitud de validación puede superar a la de entrenamiento. No es un error: durante el entrenamiento *Dropout* está activo y las métricas se promedian mientras el modelo todavía mejora, mientras que la validación se mide con el modelo completo al final de la época. Lo importante es la tendencia de `val_loss`: cuando deja de bajar, empieza el sobreajuste.

8. Evaluación final

8.1. El mejor modelo frente al conjunto de prueba

`EarlyStopping` sin `restore_best_weights` deja el modelo de la **última** época. `ModelCheckpoint` guardó el de menor `val_loss`, y ese es el que se evalúa, una sola vez, en prueba.

```
best = keras.models.load_model(best_path)
test_loss, test_acc, test_f1 = best.evaluate(exam_stream, verbose=0)
print(f"prueba -> perdida={test_loss:.4f}  exactitud={test_acc:.4f}  "
      f"F1 macro={test_f1:.4f}")
```

```
prueba -> perdida=0.0542  exactitud=0.9845  F1 macro=0.9844
```

8.2. Predicciones y matriz de confusión

```
exam_logits = best.predict(exam_stream, verbose=0)
y_pred = exam_logits.argmax(axis=1)
y_true = np.concatenate([lab.numpy().argmax(axis=1) for _, lab in exam_stream])
confusion = tf.math.confusion_matrix(y_true, y_pred,
```

```
                                        num_classes=n_classes).numpy()
print("aciertos:", int(np.trace(confusion)), "de", int(confusion.sum()))
```

```
aciertos: 9845 de 10000
```

```
fig, ax = plt.subplots(figsize=(5.0, 4.6))
ax.imshow(np.log1p(confusion), cmap="Purples")
for i in range(n_classes):
    for j in range(n_classes):
        if confusion[i, j]:
            ax.text(j, i, confusion[i, j], ha="center", va="center",
                    fontsize=6.5,
                    color="white" if i == j else "black")
ax.set_xticks(range(n_classes)); ax.set_yticks(range(n_classes))
ax.set_xlabel("predicción"); ax.set_ylabel("verdadero")
plt.tight_layout(); plt.show()
```

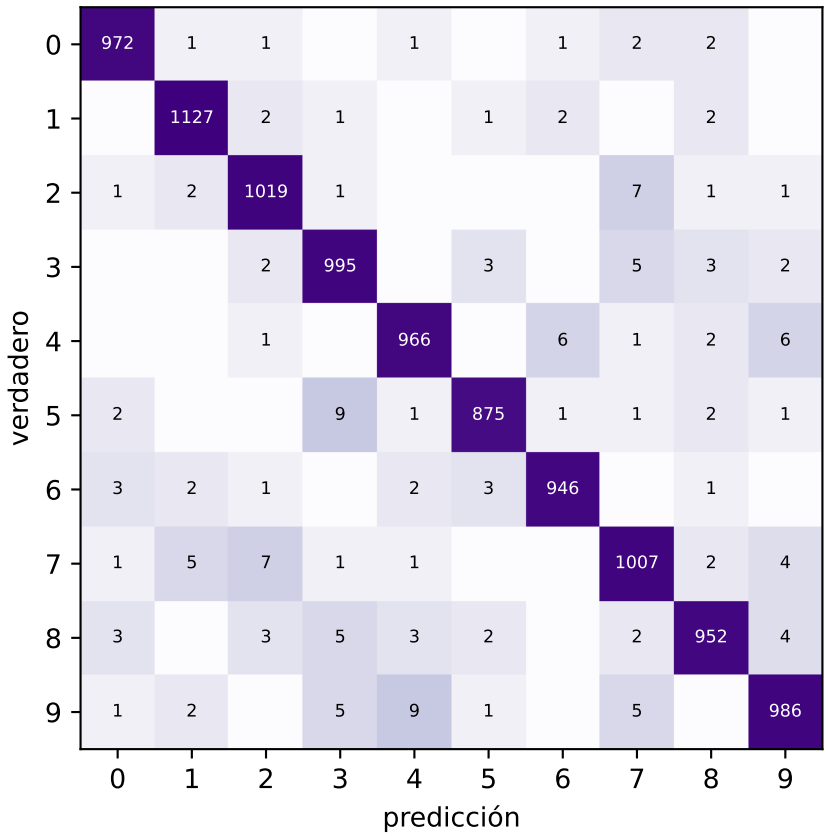


Figura 3: Matriz de confusión en prueba. Filas: clase verdadera; columnas: clase predicha.

La diagonal contiene los aciertos. Fuera de ella, la celda (i, j) cuenta cuántos dígitos i se clasificaron como j. Se usó una escala logarítmica de color, porque si no la diagonal ocultaría los errores.

8.3. Métricas por clase y confusiones frecuentes

```
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred, digits=3))

off_diag = confusion.copy()
np.fill_diagonal(off_diag, 0)
top = np.dstack(np.unravel_index(np.argsort(off_diag.ravel())[::-1][:5],
                                 off_diag.shape))[0]
print("confusiones mas frecuentes (verdadero -> predicho):")
for i, j in top:
    print(f"  {i} -> {j}: {off_diag[i, j]} casos")
```

```
              precision    recall  f1-score   support

           0      0.989     0.992     0.990       980
           1      0.989     0.993     0.991      1135
           2      0.984     0.987     0.985      1032
           3      0.978     0.985     0.982      1010
           4      0.983     0.984     0.983       982
           5      0.989     0.981     0.985       892
           6      0.990     0.987     0.989       958
           7      0.978     0.980     0.979      1028
           8      0.984     0.977     0.981       974
           9      0.982     0.977     0.980      1009

    accuracy                          0.985     10000
   macro avg      0.985     0.984     0.984     10000
weighted avg      0.985     0.985     0.984     10000

confusiones mas frecuentes (verdadero -> predicho):
  9 -> 4: 9 casos
  5 -> 3: 9 casos
  2 -> 7: 7 casos
  7 -> 2: 7 casos
  4 -> 9: 6 casos
```

8.4. Análisis de errores

Las métricas dicen **cuánto** se equivoca el modelo; los ejemplos dicen **por qué**. Se muestran los errores en los que el modelo estaba más seguro.

```
exam_images = np.concatenate([img.numpy() for img, _ in exam_stream])
probs = tf.nn.softmax(exam_logits).numpy()
wrong = np.flatnonzero(y_pred != y_true)
worst = wrong[np.argsort(probs[wrong, y_pred[wrong]])[::-1][:10]]
fig, axes = plt.subplots(2, 5, figsize=(7.2, 3.2))
for ax, idx in zip(axes.ravel(), worst):
    ax.imshow(exam_images[idx, ..., 0], cmap="gray_r")
    ax.set_title(f"{y_true[idx]}→{y_pred[idx]} ({probs[idx, y_pred[idx]]:.2f})",
                 fontsize=8)
    ax.axis("off")
```

```
plt.tight_layout(); plt.show()
```

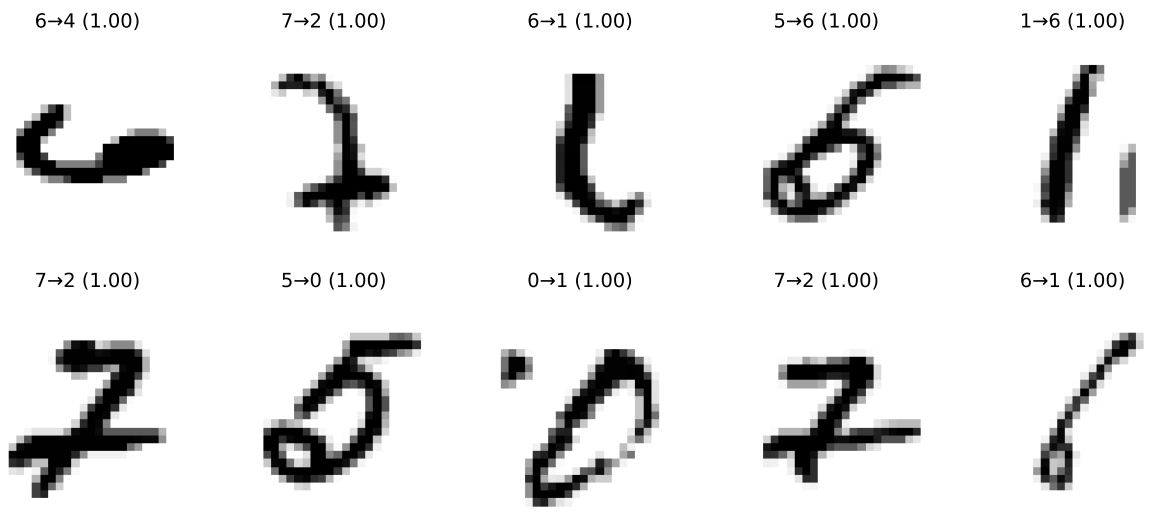


Figura 4: Errores con mayor confianza: verdadero → predicho (probabilidad asignada).

Muchos de estos errores son dígitos ambiguos incluso para una persona: trazos incompletos, formas inusuales o inclinaciones fuertes. Si el modelo comete errores **seguros** sobre dígitos claros, hay que revisar el preprocesamiento o la arquitectura. En cambio, los errores sobre ejemplos ambiguos son una cota natural del problema.

9. Dimensiones de tensores y flujo de datos

B es el tamaño del lote (`BATCH = 128`). Con 51 000 imágenes de entrenamiento hay $51\,000 = 398 \cdot 128 + 56$, es decir, 399 lotes por época, y el último tiene $B = 56$. Validación (9 000 imágenes) forma 71 lotes y prueba (10 000), 79. Los parámetros no dependen de B.

Etapa	Shape	Significado de cada dimensión	Parámetros
Imagen cruda (TFDS)	$(28, 28, 1)$, `uint8`	Alto × ancho × canal (gris), valores 0–255	—
Lote tras `prepare`	$(B, 28, 28, 1)$, `float32`	B imágenes con píxeles en $[0, 1]$	—
`Flatten`	$(B, 784)$	$28 \cdot 28 \cdot 1 = 784$ píxeles en un vector	0
`densa_1` (sin sesgo)	$(B, 256)$	256 combinaciones de píxeles	$784 \cdot 256 = 200\,704$
`bn_1`	$(B, 256)$	Las mismas 256, normalizadas	$4 \cdot 256 = 1\,024$ (512 entrenables)
`relu_1`, `dropout_1`	$(B, 256)$	No cambian la forma	0
`densa_2` (sin sesgo)	$(B, 128)$	128 rasgos de nivel superior	$256 \cdot 128 = 32\,768$
`bn_2`	$(B, 128)$	Normalización	$4 \cdot 128 = 512$ (256 entrenables)
`logits`	$(B, 10)$	Una puntuación por dígito	$128 \cdot 10 + 10 = 1\,290$

Etapa	Shape	Significado de cada dimensión	Parámetros
Etiqueta *one-hot*	$(B, 10)$	Un 1 en la columna del dígito verdadero	—
Pérdida	$()$	Promedio ponderado por `class_weight`	—

Total: 236 298 parámetros, de los cuales 235 530 son entrenables. Los 768 restantes, $2 \cdot 256 + 2 \cdot 128$, son las medias y varianzas móviles de las dos capas de normalización.

Ejemplo de interpretación. Un lote completo de 128 imágenes ocupa $128 \cdot 28 \cdot 28 = 100\,352$ valores. `Flatten` no cambia esa cantidad, solo la reorganiza en $(128, 784)$. `densa_1` multiplica $(128, 784) \times (784, 256)$ y produce $128 \cdot 256 = 32\,768$ activaciones. Al final, `predict` sobre las 10 000 imágenes de prueba devuelve una matriz $(10\,000, 10)$, y `argmax(axis=1)` la reduce a un vector $(10\,000,)$ de clases predichas.

```
xb, yb = next(iter(fit_stream))
print("lotes:", len(list(fit_stream)), len(list(check_stream)),
      len(list(exam_stream)))
print(f"{'etapa':<14}{'shape':>20}{'parametros':>12}")
print(f"{'entrada':<14}{str(tuple(xb.shape)):>20}{'-':>12}")
h_t = xb
for layer in best.layers[1:]:
    h_t = layer(h_t, training=False)
    print(f"{layer.name:<14}{str(tuple(h_t.shape)):>20}"
          f"{layer.count_params():>12}")
print(f"{'etiqueta':<14}{str(tuple(yb.shape)):>20}")
trainable = sum(int(np.prod(w.shape)) for w in best.trainable_weights)
print("total:", best.count_params(), "| entrenables:", trainable,
      "| logits de prueba:", exam_logits.shape, "-> clases:", y_pred.shape)
```

```
lotes: 399 71 79
etapa                         shape  parametros
entrada           (128, 28, 28, 1)           -
flatten                 (128, 784)           0
densa_1                 (128, 256)      200704
bn_1                    (128, 256)        1024
relu_1                  (128, 256)           0
dropout_1               (128, 256)           0
densa_2                 (128, 128)       32768
bn_2                    (128, 128)         512
relu_2                  (128, 128)           0
dropout_2               (128, 128)           0
logits                   (128, 10)        1290
etiqueta                 (128, 10)
total: 236298 | entrenables: 235530 | logits de prueba: (10000, 10) -> clases: (10000,)
```

10. Hiperparámetros del modelo

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Partición	`train[:85%]` / `train[85%:]` / `test`	Ajuste, selección y evaluación final	Más validación da decisiones más estables, con menos datos para ajustar
Tamaño de lote	128	Imágenes por paso	Mayor: menos pasos por época y gradiente más estable. Menor: más ruido y más pasos
Búfer de `shuffle`	10 000	Calidad de la mezcla	Menor que el conjunto: mezcla solo local. Mayor: más memoria
Anchuras ocultas	256 y 128	Capacidad	Más anchura: más parámetros y más riesgo de memorizar
Dropout	0.25 en cada bloque	Regularización	Mayor: más regularización y más diferencia entre la exactitud de entrenamiento y la de validación
`use_bias`	`False` en las capas densas ocultas	Evita redundancia con β de BatchNorm	Con sesgo, el resultado es equivalente con más parámetros
Tasa de aprendizaje	$2 \cdot 10^{-3}$ (Adam)	Tamaño del paso	Muy alta: inestabilidad. Muy baja: convergencia lenta
Épocas	25 como máximo	Presupuesto	Con *early stopping*, actúa como tope
`EarlyStopping`	`patience=4` sobre `val_loss`	Corta si no hay mejora	Mayor: más tolerancia a mesetas
Programador de tasa	`ReduceLROnPlateau`: `factor=0.3`, `patience=2`, `min_lr=1e-5`	Afina la tasa al estancarse	Un factor menor da recortes más fuertes
Pesos de clase	$w_c = N/(Kn_c)$, entre 0.89 y 1.11	Compensan el desbalance	En datos balanceados, efecto pequeño
Semillas	1234 (global), 99 (mezcla)	Reproducibilidad	Cambian el resultado concreto, no el método

En términos de aprendizaje, la tasa, el lote y `ReduceLROnPlateau` gobiernan la **convergencia**. *Dropout*, la anchura y el número de épocas gobiernan el **sobreajuste**. Los pesos de clase controlan cómo se **reparte el error** entre clases.

11. Errores frecuentes

1. **Calcular estadísticas con validación o prueba.** Los pesos de clase, la normalización y cualquier otra cifra se calculan solo con entrenamiento.

2. **Mezclar formatos de etiqueta.** La entropía cruzada categórica espera etiquetas *one-hot*, y su versión *sparse* espera enteros (§3.2). Usar la equivocada produce un error de forma o, peor, métricas sin sentido.
3. **Olvidar `from_logits=True`** cuando la última capa no tiene softmax. La pérdida trataría los logits como si fueran probabilidades.
4. **`cache()` después de `shuffle()`.** La caché fija el primer orden mezclado y todas las épocas repiten la misma secuencia.
5. **Evaluar el modelo de la última época sin querer.** Si no se restaura el mejor modelo, se reporta uno que quizá ya sobreajustó.
6. **Juzgar solo por la exactitud.** Con clases desbalanceadas, una exactitud alta puede ocultar una clase que nunca se acierta. El F1 macro y la matriz de confusión lo revelan.

12. Ejercicios

1. **Softmax a mano.** Para los logits $(1,\ 1,\ 1)$, ¿qué probabilidades da softmax? ¿Y para $(101,\ 101,\ 101)$? ¿Qué propiedad muestra el resultado?
2. **Pesos de clase.** Con $N = 600$, $K = 3$ y conteos $(400, 150, 50)$, calcula w_c.
3. **Sin normalización.** Elimina las capas `BatchNormalization` (y devuelve el sesgo a las capas densas). Compara cuántas épocas tarda en estabilizarse la pérdida de validación.
4. **Etiquetas enteras.** Reescribe `prepare` y la compilación para usar `SparseCategoricalCrossentropy` sin *one-hot*. ¿Qué métrica de exactitud debes usar?
5. **Desbalance artificial.** Elimina el 90 % de los ejemplos del dígito 5 del entrenamiento y entrena con y sin `class_weight`. Compara la sensibilidad de esa clase.
6. **Convolución.** Sustituye el primer bloque denso por `Conv2D(32, 3)` → `MaxPooling2D` → `Flatten`. ¿Cuántos parámetros tiene ahora el modelo y cómo cambia la exactitud?

12.1. Respuestas orientativas

1. En ambos casos $(1/3,\ 1/3,\ 1/3)$: softmax es invariante a sumar una constante a todos los logits. Las implementaciones aprovechan esa propiedad y restan máx z para evitar desbordamientos.
2. $w = 600/(3 \cdot 400) = 0.5$; $600/(3 \cdot 150) = 1.333$; $600/(3 \cdot 50) = 4.0$.
3. Sin normalización, la red suele necesitar más épocas o una tasa de aprendizaje menor para estabilizarse. Con dos capas la diferencia es moderada.
4. `prepare` devuelve `lab` sin `one_hot`, se usa `SparseCategoricalCrossentropy(from_logits=True)` y la métrica `SparseCategoricalAccuracy`. `F1Score` requiere etiquetas *one-hot*.
5. Sin pesos, la sensibilidad del 5 cae. Con pesos se recupera en parte, normalmente a costa de algo de precisión en esa clase.
6. La convolución comparte pesos espacialmente. Con 32 filtros de 3×3 añade solo 320 parámetros, aunque la capa densa que sigue al `Flatten` puede seguir siendo grande. La exactitud suele subir por encima del 99 %.

13. Glosario

Término	Significado
Pipeline	Cadena completa de datos, modelo, entrenamiento y evaluación
TFDS	Catálogo de conjuntos de datos listos para `tf.data`

Término	Significado
One-hot	Vector con un 1 en la posición de la clase y 0 en las demás
Softmax	Función que convierte logits en probabilidades que suman 1
Pesos de clase	Ponderación de la pérdida según la frecuencia de cada clase
F1 macro	Promedio simple del F1 de cada clase
Matriz de confusión	Tabla de conteos de clase verdadera frente a clase predicha
Checkpoint	Copia en disco del mejor modelo según una métrica de validación

14. Conclusiones académicas

Qué se aprendió. Un proyecto de clasificación no es solo un modelo, sino una cadena:

- datos divididos sin fugas;
- estadísticas calculadas solo con entrenamiento;
- un flujo `tf.data` ordenado con criterio;
- una arquitectura con normalización y regularización;
- *callbacks* que automatizan la selección del modelo;
- una evaluación final, única y desagregada por clase.

Qué demostraron los experimentos.

- El modelo denso de 236 298 parámetros alcanzó una exactitud de 0.9845 y un F1 macro de 0.9844 sobre las 10 000 imágenes de prueba (9845 aciertos).
- El entrenamiento completó 25 épocas sin activar `EarlyStopping`. La menor pérdida de validación se registró en la época 21, y ese fue el modelo que guardó `ModelCheckpoint` y que se evaluó.
- `ReduceLROnPlateau` bajó la tasa de $2 \cdot 10^{-3}$ a 5.4e-05.
- Las confusiones más frecuentes fueron 9→4 (9 casos) y 5→3 (9 casos), pares de dígitos con trazos parecidos.

Cómo interpretar los resultados. Que la exactitud y el F1 macro sean casi iguales indica que ninguna clase queda claramente rezagada. Varios de los errores con mayor confianza de la figura de errores parecen ambiguos también para una persona. Parte del error restante puede deberse al propio problema y no al modelo. Al final del historial, la pérdida de entrenamiento queda muy por debajo de la de validación, y la de validación ya no mejora. Eso es compatible con un sobreajuste incipiente. Por eso importa evaluar el mejor punto de control y no el modelo de la última época.

Limitaciones.

- Una sola ejecución, con una semilla y una partición.
- Una red densa que ignora la estructura espacial de la imagen.
- MNIST es un conjunto limpio y casi balanceado, así que el efecto de los pesos de clase no se pudo evaluar aquí.
- No se midió la incertidumbre de las métricas, por ejemplo con *bootstrap*.

Qué conviene profundizar.

- Redes convolucionales y aumento de datos.
- Conjuntos de datos más difíciles, como Fashion-MNIST o CIFAR-10.

- Desbalance real de clases (ejercicio 5).
- Intervalos de confianza para las métricas y calibración de las probabilidades.

Relación con el *Deep Learning*. El mismo *pipeline* (carga eficiente, normalización, lotes, *callbacks*, puntos de control, evaluación por clase y análisis de errores) es el que se usa para entrenar modelos de visión mucho mayores. Lo que cambia es la arquitectura. La disciplina de no filtrar información y de evaluar una sola vez en prueba es la misma.

15. Referencias

- Hortúa, H. J. *Deep Learning I — Clase 02: `Pipeline_TensorFlow.ipynb`*. Universidad El Bosque. Referencia conceptual de esta guía (enlaces en el recuadro de procedencia).
- LeCun, Y., Cortes, C. y Burges, C. J. C. *The MNIST database of handwritten digits*.
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 6 y 11.
- Ioffe, S. y Szegedy, C. (2015). *Batch Normalization*. ICML.
- Documentación: `tf.data` https://www.tensorflow.org/guide/data · TFDS https://www.tensorflow.org/datasets · Keras https://keras.io/api/callbacks/